

Soc 111 - Week 5 Thursday - Homophily and ergm

GR

2024-09-05

```
#install.packages("statnet")
library(ergm)
library(igraph)
library(ggnetwork)
library(tidyverse)
#library(intergraph)
```

Homophily

- 173 example
- homophily vs elective affinity
- entropy vs isolation/exposure

Lin and Lundquist (2013)

Lin, Ken-Hou, and Lundquist, Jennifer. 2013. "Mate Selection in Cyberspace: The Intersection of Race, Gender, and Education." *American Journal of Sociology* 119:183–215.

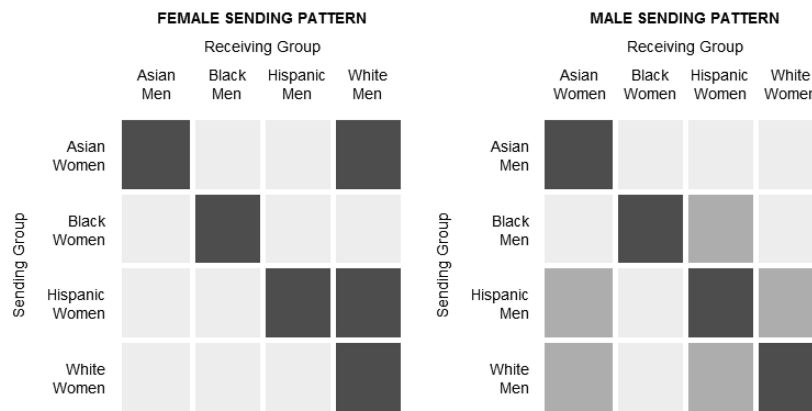


Figure 1: Fig. 1. Predicted odds ratios of sending an initial message (darker cells represent higher probabilities).

Lundquist and Lin (2015)

Lundquist, Jennifer H., and Ken-Hou Lin. 2015. "Is Love (Color) Blind? The Economy of Race among Gay and Straight Daters." *Social Forces* 93(4):1423–49. doi: 10.1093/sf/sov008.

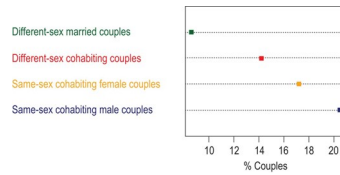


Figure 2: Figure 1: 2000 Census: Percentage of age 20–34 same-sex and different-sex interracial households

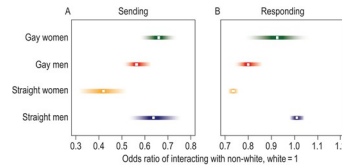


Figure 3: Figure 2. Predicted probabilities with confidence intervals of interacting with out-group relative to white in-group

Exponential random graph models

Lightly adapted from <https://badhessian.org/2012/09/lessons-on-exponential-random-graph-modeling-from-greys-anatomy-hook-ups>

- Import GA edgelist.
- Import attributes file.
- Make a graph object.

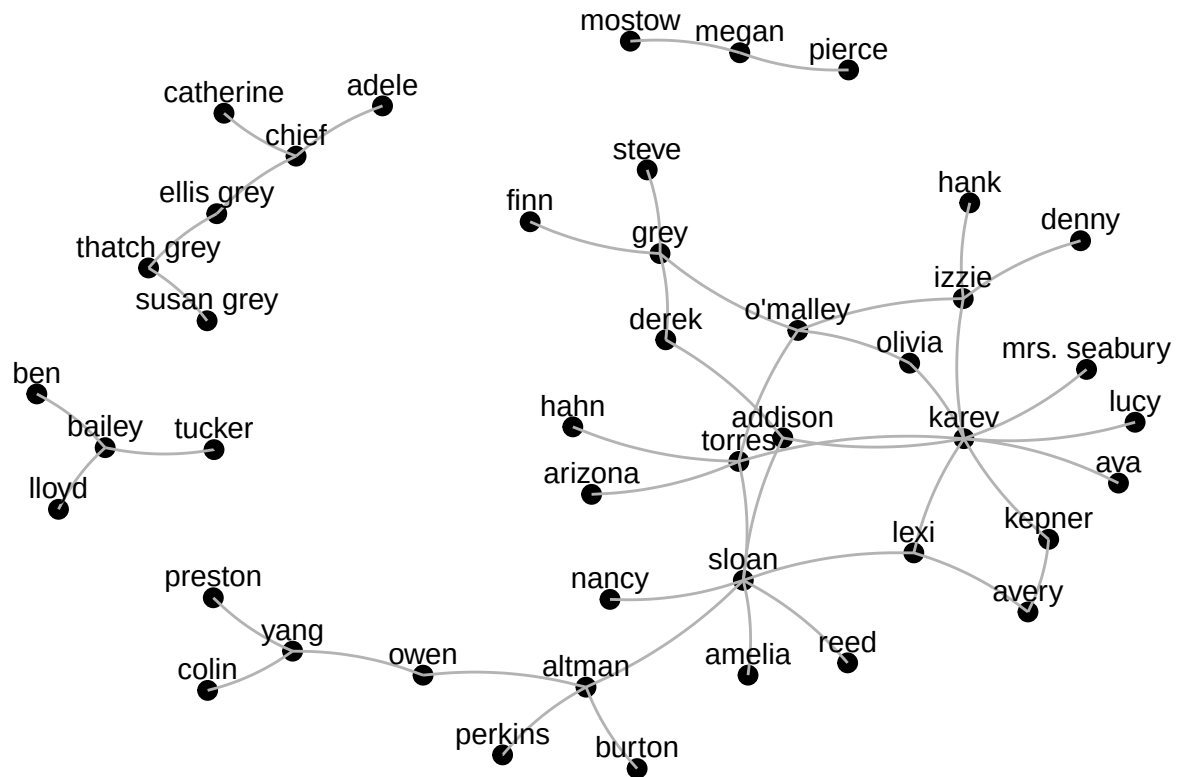
```
# # create in igraph format
ga.edges.df <- read_csv("ga_edgelist.csv", col_names = FALSE)
glimpse(ga.edges.df)
ga.attrs.df <- read_tsv("GreysAnatomy_attributes.tsv")
glimpse(ga.attrs.df)
ga.g <- graph_from_data_frame(ga.edges.df,
#                               vertices = ga.attrs.df,
#                               directed = FALSE) %>%
#   igraph::simplify()
# V(ga.g)$color <- ifelse( V(ga.g)$sex == "F", "pink", "blue")
# vertex_attr(ga.g)
# # convert to statnet/network/ergm format
ga.network <- intergraph::asNetwork(ga.g)
write_rds(ga.g, "ga.g.rds")
write_rds(ga.network, "ga.network.rds")
```

```
ga.g <- read_rds("ga.g.rds")
ga.network <- read_rds("ga.network.rds")
```

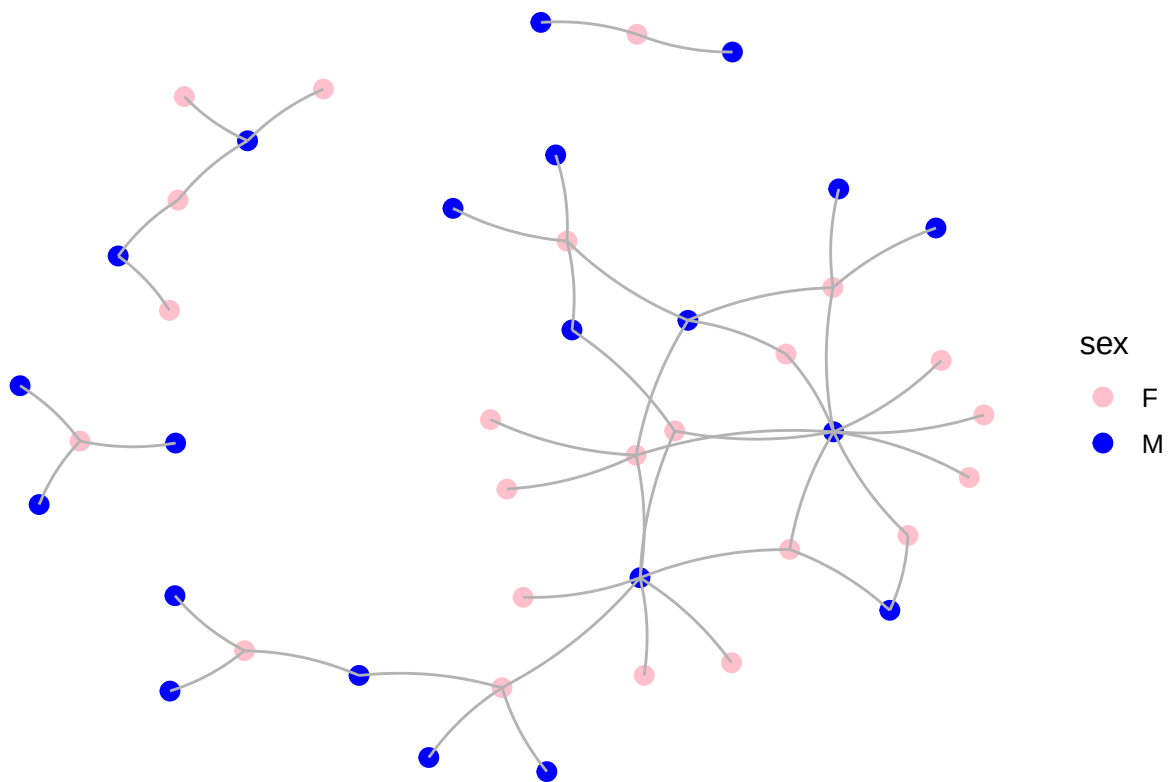
```
ga.gg <- ga.g %>% ggnetwork()
```

```
ga.gg %>%
  ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_nodes(size=3) +
  geom_edges(color="gray70", curvature = 0.1) +
```

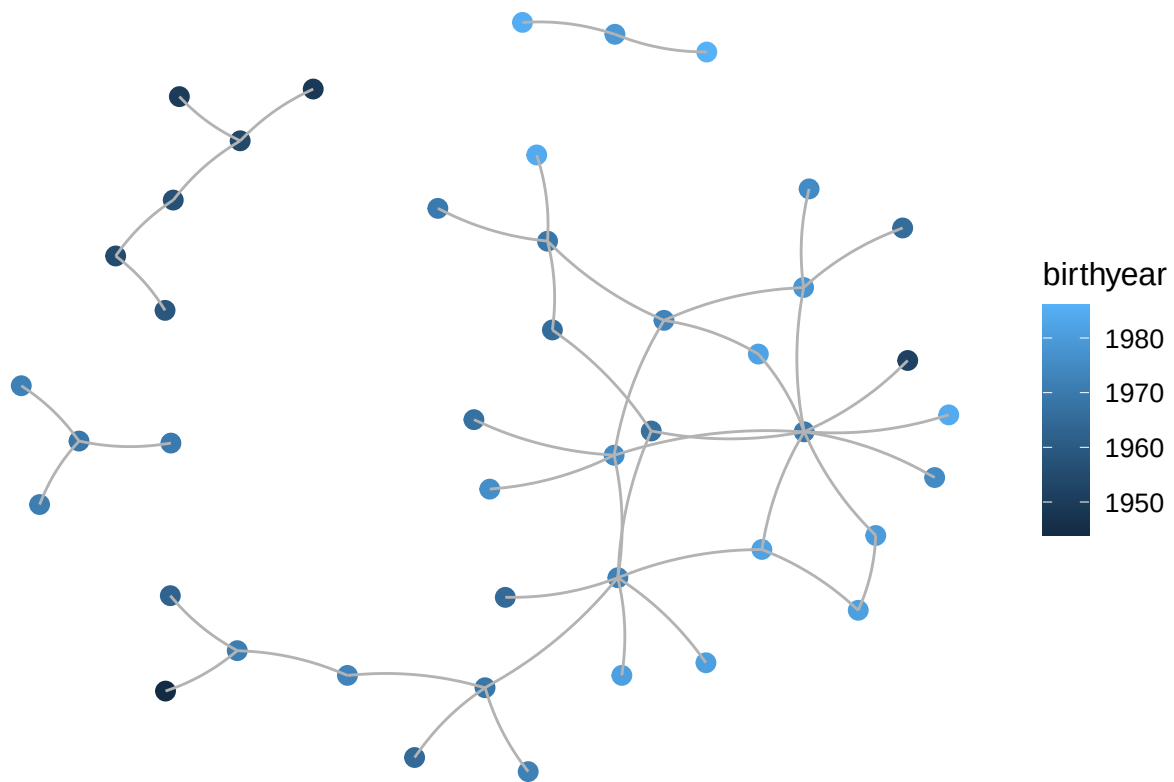
```
geom_nodetext(aes(label = name), nudge_y = 0.03) +  
theme_blank()
```



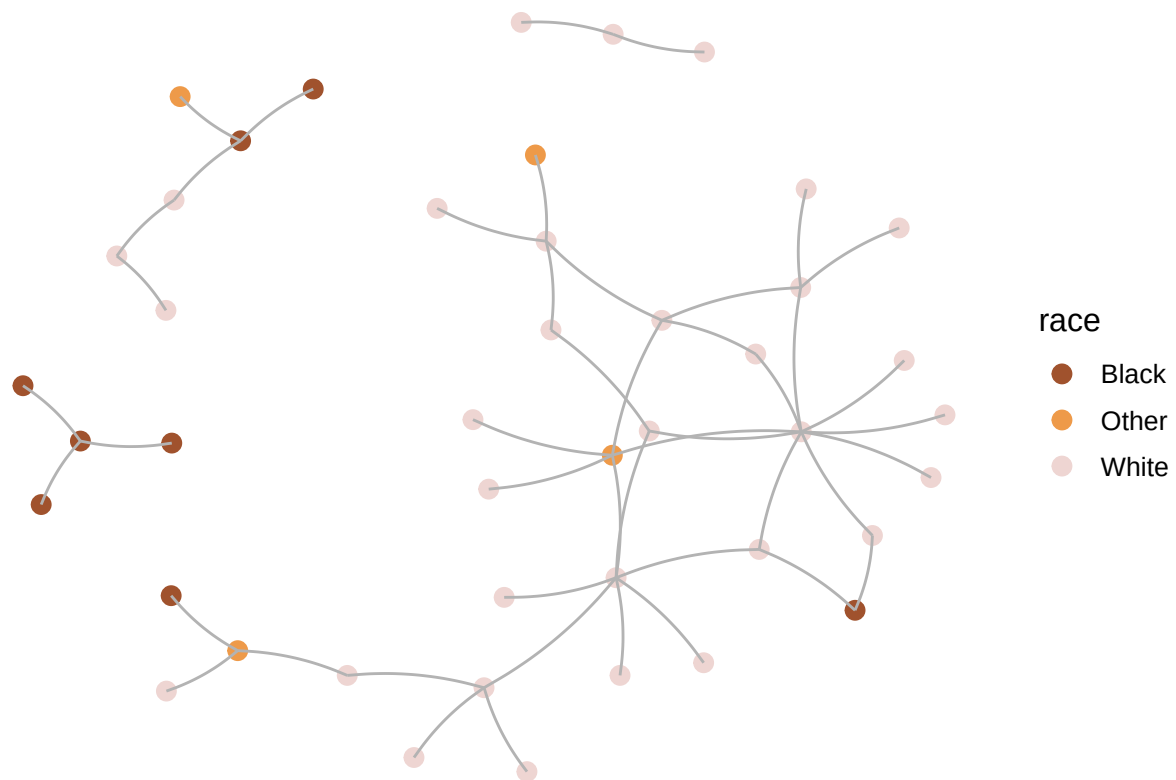
```
ga.gg %>%  
ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +  
geom_nodes(aes(color=sex), size=3) +  
geom_edges(color="gray70", curvature = 0.1) +  
scale_color_manual(values=c("pink", "blue")) +  
theme_blank()
```



```
ga.gg %>%
  ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_nodes(aes(color=birthyear), size=3) +
  geom_edges(color="gray70", curvature = 0.1) +
  scale_fill_distiller() +
  theme_blank()
```



```
ga.gg %>%
  ggplot(aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_nodes(aes(color=race), size=3) +
  geom_edges(color="gray70", curvature = 0.1) +
  scale_color_manual(values=c("sienna","tan2","mistyrose2")) +
  theme_blank()
```



```
ga.ergm.1 <- ergm(ga.network ~ edges + nodematch("sex"))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.
```

```
## Evaluating the predictor and response matrix.
```

```
## Maximizing the pseudolikelihood.
```

```
## Finished MPLE.
```

```
## Evaluating log-likelihood at the estimate.
```

```
summary(ga.ergm.1)
```

```
## Call:
```

```
## ergm(formula = ga.network ~ edges + nodematch("sex"))
```

```
##
```

```
## Maximum Likelihood Results:
```

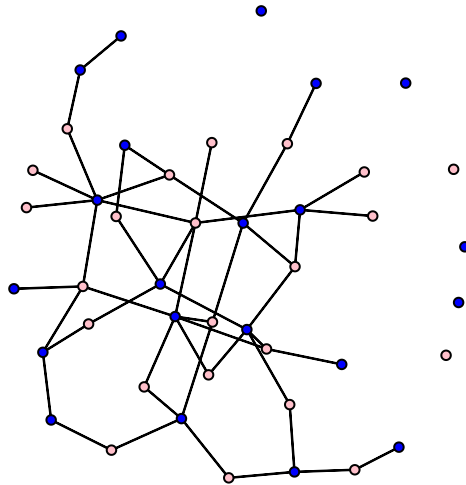
```
##
```

```
##           Estimate Std. Error MCMC % z value Pr(>|z|)
```

```
## edges      -2.3003    0.1581    0 -14.547  <1e-04 ***
```

```
## nodematch.sex -3.1399      0.7260      0 -4.325 <1e-04 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 1311.4 on 946 degrees of freedom
## Residual Deviance: 320.5 on 944 degrees of freedom
##
## AIC: 324.5 BIC: 334.2 (Smaller is better. MC Std. Err. = 0)
```

```
ga.ergm.1 %>%
  simulate() %>%
  plot(vertex.col=c("blue", "pink") [ 1+(network::get.vertex.attribute(ga.network, "sex")==="F")])
```



```
ga.ergm.1 %>% simulate() %>% degreedist() # degree distribution in simulated data
```

```
## degree0 degree1 degree2 degree3 degree4 degree5
##      6      13      8      10      6      1
```

```
ga.g %>% degree() %>% table() # degree dist in actual data
```

```
## .
## 1 2 3 4 5 7 9
## 24 8 5 4 1 1 1
```

```

ga.ergm.2 <- ergm(ga.network ~ edges + nodematch("sex") + degree(1))

## Starting maximum pseudolikelihood estimation (MPLE):

## Obtaining the responsible dyads.

## Evaluating the predictor and response matrix.

## Maximizing the pseudolikelihood.

## Finished MPLE.

## Starting Monte Carlo maximum likelihood estimation (MCMLE):

## Iteration 1 of at most 60:

## Warning: 'glpk' selected as the solver, but package 'Rglpk' is not available;
## falling back to 'lpSolveAPI'. This should be fine unless the sample size and/or
## the number of parameters is very big.

## Optimizing with step length 0.9363.

## The log-likelihood improved by 4.8019.

## Estimating equations are not within tolerance region.

## Iteration 2 of at most 60:

## Optimizing with step length 0.7869.

## The log-likelihood improved by 3.1819.

## Estimating equations are not within tolerance region.

## Iteration 3 of at most 60:

## Optimizing with step length 1.0000.

## The log-likelihood improved by 0.1097.

## Estimating equations are not within tolerance region.

## Iteration 4 of at most 60:

## Optimizing with step length 1.0000.

## The log-likelihood improved by 0.0016.

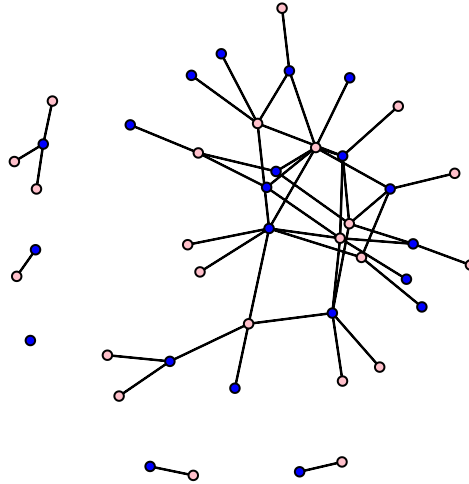
```

```
## Convergence test p-value: 0.0272. Not converged with 99% confidence; increasing sample size.
## Iteration 5 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0571.
## Convergence test p-value: 0.4363. Not converged with 99% confidence; increasing sample size.
## Iteration 6 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0874.
## Convergence test p-value: 0.3668. Not converged with 99% confidence; increasing sample size.
## Iteration 7 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0092.
## Convergence test p-value: < 0.0001. Converged with 99% confidence.
## Finished MCMLE.
## Evaluating log-likelihood at the estimate. Fitting the dyad-independent submodel...
## Bridging between the dyad-independent submodel and the full model...
## Setting up bridge sampling...
## Using 16 bridges: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 .
## Bridging finished.
##
## This model was fit using MCMC. To examine model diagnostics and check
## for degeneracy, use the mcmc.diagnostics() function.
```

```
summary(ga.ergm.2)
```

```
## Call:
## ergm(formula = ga.network ~ edges + nodematch("sex") + degree(1))
##
## Monte Carlo Maximum Likelihood Results:
##
##           Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -1.4327    0.2443     0  -5.864  <1e-04 ***
## nodematch.sex   -3.1766    0.7015     0  -4.528  <1e-04 ***
## degree1         2.0679    0.4437     0   4.661  <1e-04 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 1311.4 on 946 degrees of freedom
## Residual Deviance: 295.6 on 943 degrees of freedom
##
## AIC: 301.6 BIC: 316.2 (Smaller is better. MC Std. Err. = 0.1935)
```

```
ga.ergm.2 %>%
  simulate() %>%
  plot(vertex.col=c("blue","pink") [ 1+(network::get.vertex.attribute(ga.network, "sex")==="F")])
```



```
ga.ergm.2 %>% simulate() %>% degreedist() # degree distribution in simulated data
```

```
## degree1 degree2 degree3 degree4 degree5 degree6
##      30       4       6       1       2       1
```

```
ga.ergm.3 <- ergm(ga.network ~ edges +
  nodematch("sex") +
  degree(1) +
  absdiff("birthyear"))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
```

```
## Obtaining the responsible dyads.
```

```
## Evaluating the predictor and response matrix.
```

```
## Maximizing the pseudolikelihood.
```

```
## Finished MPLE.
```

```
## Starting Monte Carlo maximum likelihood estimation (MCMLE):
```

```
## Iteration 1 of at most 60:
```

```

## Optimizing with step length 0.9417.
## The log-likelihood improved by 3.3304.
## Estimating equations are not within tolerance region.
## Iteration 2 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 1.3474.
## Estimating equations are not within tolerance region.
## Iteration 3 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.1930.
## Estimating equations are not within tolerance region.
## Iteration 4 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.1684.
## Estimating equations are not within tolerance region.
## Iteration 5 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0737.
## Convergence test p-value: 0.7983. Not converged with 99% confidence; increasing sample size.
## Iteration 6 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0223.
## Convergence test p-value: 0.0549. Not converged with 99% confidence; increasing sample size.
## Iteration 7 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0117.
## Convergence test p-value: 0.0012. Converged with 99% confidence.
## Finished MCMLE.
## Evaluating log-likelihood at the estimate. Fitting the dyad-independent submodel...
## Bridging between the dyad-independent submodel and the full model...
## Setting up bridge sampling...
## Using 16 bridges: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 .
## Bridging finished.
##
## This model was fit using MCMC. To examine model diagnostics and check
## for degeneracy, use the mcmc.diagnostics() function.

```

```
summary(ga.ergm.3)
```

```
## Call:
## ergm(formula = ga.network ~ edges + nodematch("sex") + degree(1) +
##       absdiff("birthyear"))
##
## Monte Carlo Maximum Likelihood Results:
##
##               Estimate Std. Error MCMC % z value Pr(>|z|)
## edges           -0.47477    0.29142     0  -1.629    0.103
## nodematch.sex    -3.19555    0.78081     0  -4.093   <1e-04 ***
## degree1          1.99502    0.44151     0   4.519   <1e-04 ***
## absdiff.birthyear -0.12346    0.02939     0  -4.201   <1e-04 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 1311  on 946  degrees of freedom
## Residual Deviance:  270  on 942  degrees of freedom
##
## AIC: 278  BIC: 297.4  (Smaller is better. MC Std. Err. = 0.3386)
```

```
ga.ergm.4 <- ergm(ga.network ~ edges +
  nodematch("sex") +
  degree(1) +
  absdiff("birthyear") +
  nodematch("race"))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
## Obtaining the responsible dyads.
## Evaluating the predictor and response matrix.
## Maximizing the pseudolikelihood.
## Finished MPLE.
## Starting Monte Carlo maximum likelihood estimation (MCMLE):
## Iteration 1 of at most 60:
## Optimizing with step length 0.8413.
## The log-likelihood improved by 2.2491.
## Estimating equations are not within tolerance region.
## Iteration 2 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.9881.
## Estimating equations are not within tolerance region.
## Iteration 3 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.1617.
## Estimating equations are not within tolerance region.
## Iteration 4 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0380.
## Convergence test p-value: 0.8805. Not converged with 99% confidence; increasing sample size.
## Iteration 5 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0498.
## Convergence test p-value: 0.6392. Not converged with 99% confidence; increasing sample size.
```

```
## Iteration 6 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0186.
## Convergence test p-value: 0.1044. Not converged with 99% confidence; increasing sample size.
## Iteration 7 of at most 60:
## Optimizing with step length 1.0000.
## The log-likelihood improved by 0.0112.
## Convergence test p-value: 0.0097. Converged with 99% confidence.
## Finished MCMLE.
## Evaluating log-likelihood at the estimate. Fitting the dyad-independent submodel...
## Bridging between the dyad-independent submodel and the full model...
## Setting up bridge sampling...
## Using 16 bridges: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 .
## Bridging finished.
##
## This model was fit using MCMC. To examine model diagnostics and check
## for degeneracy, use the mcmc.diagnostics() function.
```

```
summary(ga.ergm.4)
```

```
## Call:
## ergm(formula = ga.network ~ edges + nodematch("sex") + degree(1) +
##       absdiff("birthyear") + nodematch("race"))
##
## Monte Carlo Maximum Likelihood Results:
##
##               Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -0.97869    0.40222    0  -2.433   0.0150 *
## nodematch.sex   -3.33031    0.74881    0  -4.447   <1e-04 ***
## degree1         1.95759    0.42113    0   4.648   <1e-04 ***
## absdiff.birthyear -0.13315    0.03016    0  -4.414   <1e-04 ***
## nodematch.race    0.82591    0.37538    0   2.200   0.0278 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##      Null Deviance: 1311.4 on 946 degrees of freedom
## Residual Deviance: 264.3 on 941 degrees of freedom
##
## AIC: 274.3 BIC: 298.5 (Smaller is better. MC Std. Err. = 0.3099)
```

```
rr.edges.df <- read_csv("rogerrabbit_edges.csv", col_names = FALSE)
```

```
## Rows: 17 Columns: 2
## -- Column specification -----
## Delimiter: ","
## chr (2): X1, X2
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
rr.nodes.df <- read_csv("rogerrabbit_nodes.csv", col_names = TRUE)
```

```
## Rows: 10 Columns: 3
## -- Column specification -----
## Delimiter: ","
## chr (1): name
## dbl (2): toon, female
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
rr.g <- graph_from_data_frame(rr.edges.df,
                             vertices = rr.nodes.df,
                             directed = FALSE) %>%
  igraph::simplify()

rr.network <- intergraph::asNetwork(rr.g)

rr.ergm.1 <- ergm(rr.network ~ edges + nodematch("toon"))
```

```
## Starting maximum pseudolikelihood estimation (MPLE):
## Obtaining the responsible dyads.
## Evaluating the predictor and response matrix.
## Maximizing the pseudolikelihood.
## Finished MPLE.
## Evaluating log-likelihood at the estimate.
```

```
summary(rr.ergm.1)
```

```
## Call:
## ergm(formula = rr.network ~ edges + nodematch("toon"))
##
## Maximum Likelihood Results:
##
##              Estimate Std. Error MCMC % z value Pr(>|z|)
## edges          -0.57536    0.41667      0  -1.381    0.167
## nodematch.toon -0.04368    0.62721      0  -0.070    0.944
##
##      Null Deviance: 62.38  on 45  degrees of freedom
## Residual Deviance: 58.57  on 43  degrees of freedom
##
## AIC: 62.57  BIC: 66.18  (Smaller is better. MC Std. Err. = 0)
```